# Code Explanation

**Batch Processing ETL Pipeline Explanation**

The provided code is a batch processing ETL (Extract, Transform, Load) pipeline implemented using Apache Spark and Delta Lake. It reads customer and order data from CSV files, cleans and transforms the data, and loads it into Delta tables for further analysis.

## Overview of the Code
The ETL pipeline performs the following tasks:
- Creates a Spark session with necessary configurations
- Reads customer and order data from CSV files
- Cleans and transforms the data by removing nulls and duplicates
- Creates or updates a Slowly Changing Dimension (SCD) Type 2 table for order summary
- Creates an aggregate table for customer spend

## Step 1: Create a Spark Session
This step creates a Spark session with the necessary configurations for Delta Lake.

```
def create_spark_session():
    """Create a Spark session with necessary configurations"""
    return (SparkSession.builder
            .appName("Customer Order Processing")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

## Step 2: Read Customer and Order Data
This step reads customer and order data from CSV files using predefined schemas.

```
def read_customer_data(spark):
    """Read customer data from volume"""
    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    return (spark.read.format("csv")
            .option("header", "true")
            .schema(customer_schema)
            .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/cus
tomerdata"))

def read_order_data(spark):
    """Read order data from volume"""
    order_schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
```

```
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    return (spark.read.format("csv")
            .option("header", "true")
            .schema(order_schema)
            .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/ord
erdata"))
```

## Step 3: Clean Customer and Order Data
This step cleans the customer and order data by removing nulls and duplicates.

```
def clean_customer_data(customer_df):
    """Clean customer data by removing nulls and duplicates"""
    return (customer_df
            .filter(
                (col("CustId").isNotNull()) &
                (col("Name").isNotNull()) &
                (col("EmailId").isNotNull()) &
                (col("Region").isNotNull())
            )
            .dropDuplicates(["CustId"]))

def clean_order_data(order_df):
    """Clean order data and add TotalAmount column"""
    return (order_df
            .filter(
                (col("OrderId").isNotNull()) &
                (col("ItemName").isNotNull()) &
                (col("PricePerUnit").isNotNull()) &
                (col("Qty").isNotNull()) &
                (col("Date").isNotNull()) &
                (col("CustId").isNotNull())
            )
            .dropDuplicates(["OrderId"])
            .withColumn("TotalAmount", col("PricePerUnit") * col("Qt
y")))
```

## Step 4: Create or Update SCD Type 2 Table for Order Summary
This step creates or updates the SCD Type 2 table for order summary.

```
def create_or_update_ordersummary(spark, customer_df, order_df):
    """Create or update the ordersummary SCD Type 2 table"""
    # Create catalog and schema if they don't exist
    spark.sql("CREATE CATALOG IF NOT EXISTS gen_ai_poc_databrickscoe
")
    spark.sql("CREATE SCHEMA IF NOT EXISTS gen_ai_poc_databrickscoe.
sdlc_wizard")

    # Join customer and order data
    joined_data = (customer_df
                   .join(order_df, "CustId", "inner")
```

```
                .select(
                    "CustId",
                    "Name",
                    "EmailId",
                    "Region",
                    "OrderId",
                    "ItemName",
                    "PricePerUnit",
                    "Qty",
                    "Date",
                    "TotalAmount"
                ))

    # Check if the table exists
    table_exists = spark._jsparkSession.catalog().tableExists("gen_a
i_poc_databrickscoe", "sdlc_wizard", "ordersummary")

    # For initial load
    if not table_exists:
        (joined_data
         .withColumn("IsActive", lit(True))
         .withColumn("StartDate", current_timestamp())
         .withColumn("EndDate", lit(None).cast(TimestampType()))
         .withColumn("ChangeHash", expr("md5(concat(Name, EmailId, R
egion))"))
         .write
         .format("delta")
         .mode("overwrite")
         .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.ordersum
mary"))
        return

    # For updates (SCD Type 2 implementation)
    delta_table = DeltaTable.forName(spark, "gen_ai_poc_databricksco
e.sdlc_wizard.ordersummary")
    existing_data = delta_table.toDF()

    # Identify changes in customer data
    customer_changes = (customer_df
                        .withColumn("ChangeHash", expr("md5(concat(Na
me, EmailId, Region))"))
                        .alias("new")
                        .join(
                            existing_data.select("CustId", "ChangeHas
h").alias("old"),
                            col("new.CustId") == col("old.CustId"),
                            "left"
                        )
                        .where(
                            (col("new.ChangeHash") != col("old.Change
Hash")) |
                            col("old.ChangeHash").isNull()
                        )
                        .select(
                            col("new.CustId").alias("CustId"),
                            col("new.ChangeHash").alias("NewHash")
                        ))

    # Mark existing records as inactive
```

```
    delta_table.alias("existing").merge(
        customer_changes.alias("updates"),
        "existing.CustId = updates.CustId"
    ).whenMatched().updateExpr({
        "IsActive": "false",
        "EndDate": "current_timestamp()"
    }).execute()

    # Insert new active records
    new_records = (joined_data
                   .join(customer_changes, "CustId", "inner")
                   .withColumn("IsActive", lit(True))
                   .withColumn("StartDate", current_timestamp())
                   .withColumn("EndDate", lit(None).cast(TimestampTyp
e()))
                   .withColumn("ChangeHash", expr("md5(concat(Name, E
mailId, Region))")))

    new_records.write.format("delta").mode("append").saveAsTable("ge
n_ai_poc_databrickscoe.sdlc_wizard.ordersummary")
```

## Step 5: Create Aggregate Table for Customer Spend
This step creates an aggregate table for customer spend.

```
def create_customer_aggregate_spend(spark):
    """Create the customeraggregatespend table with aggregated data"
""
    # Create table if not exists
    spark.sql("""
    CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.
customeraggregatespend (
        Name STRING,
        TotalAmount DOUBLE,
        Date DATE
    ) USING DELTA
    """)

    # Aggregate data and insert into table
    spark.sql("""
    INSERT OVERWRITE TABLE gen_ai_poc_databrickscoe.sdlc_wizard.cust
omeraggregatespend
    SELECT Name, SUM(TotalAmount) as TotalAmount, Date
    FROM gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary
    WHERE IsActive = true
    GROUP BY Name, Date
    """)
```

## Step 6: Orchestrate the ETL Process
This step orchestrates the ETL process by calling the above functions in sequence.

```
def main():
    """Main function to orchestrate the ETL process"""
    spark = create_spark_session()
```

```
    # Read source data
    customer_df = read_customer_data(spark)
    order_df = read_order_data(spark)

    # Clean data
    customer_clean = clean_customer_data(customer_df)
    order_clean = clean_order_data(order_df)

    # Create or update SCD Type 2 table
    create_or_update_ordersummary(spark, customer_clean, order_clean
)

    # Create aggregate table
    create_customer_aggregate_spend(spark)

    spark.stop()

if __name__ == "__main__":
    main()
```